WEEK 7 WARM-UP · PANDAS DATAFRAMES

CU HOCKEY BY THE NUMBERS

Ten questions about real CU Boulder club hockey data. Answer each one with a line or two of pandas.

THE DATA

Four CSV files, three CU teams

117
`cu_games`
one row per game

120
`cu_skaters`
one row per player per season

9
`cu_goalies`
one row per goalie per season

84
`cu_rosters`
height, weight, shoots, hometown

Team	Seasons	Conference
CU Men's D1	2024-25	WCHL
CU Men's D2	2023-24 and 2026-27 (in progress)	BMHC
CU Women's D1	2024-25	WWCHL

HOW IT WORKS

Load once, then answer each question

START WITH THIS CELL

```
import pandas as pd
games   = pd.read_csv("cu_hockey_data/cu_games.csv")
skaters = pd.read_csv("cu_hockey_data/cu_skaters.csv")
rosters = pd.read_csv("cu_hockey_data/cu_rosters.csv")
```

1 Pair up and read the question

2 Write one or two lines of pandas

3 Compare answers before the reveal

QUESTION 1 OF 10 · INSPECT

How many games are in the file, and how many have been played?

TRY

```
games.shape
games["status"].value_counts()
```

💡 The 2026-27 season isn't over. What do the scores of future games look like?

QUESTION 2 OF 10 · SELECT A COLUMN

What's the most penalty minutes one player racked up in a season?

TRY

```
skaters["pim"].max()
```

💡 Now find out who it was. A column is a Series, and **nlargest(1, "pim")** works on the whole DataFrame.

QUESTION 3 OF 10 · FILTER ROWS

Which players scored 20 or more points in a season?

TRY

```
skaters[skaters["points"] >= 20]
```

💡 How many are on each team? Add **["team"].value_counts()** to the result.

QUESTION 4 OF 10 · SORT

Who are CU's top 5 scorers across all three teams?

TRY

```
skaters.sort_values("points", ascending=False).head(5)
skaters.nlargest(5, "points")
```

💡 Both lines give the same answer. Which team does the number one scorer play for?

QUESTION 5 OF 10 · ADD A COLUMN

What was CU's biggest win?

TRY

```
games["goal_diff"] = games["cu_goals"] - games["opp_goals"]
games.nlargest(1, "goal_diff")
```

💡 The subtraction runs on every row at once. No loop needed, unlike last week's lists.

QUESTION 6 OF 10 · GROUP

Which team-season has the best winning percentage?

TRY

```
played = games[games["status"] != "Scheduled"].copy()
played["win"] = played["result"] == "W"
played.groupby(["team", "season"])["win"].mean()
```

⚠️ Is the winner a fair comparison? Check how many games each team has played.

QUESTION 7 OF 10 · GROUP

Do CU teams win more often at home than on the road?

TRY

```
played.groupby("home_away")["win"].mean()
```

💡 Then split it by team too: **groupby(["team", "home_away"])**. Does every team have a home-ice edge?

QUESTION 8 OF 10 · MISSING DATA

Which columns have missing values, and why are they missing?

TRY

```
games.isna().sum()
rosters.isna().sum()
```

💡 Should a missing weight be replaced with 0? What would that do to the team's average?

Are defensemen taller than forwards?

TRY

```
rosters.groupby(["team", "position_group"])["height_in"].mean().unstack()
```

💡 Why group by team as well, not only by position? Try it without **"team"** and compare.

QUESTION 10 OF 10 · MERGE TWO TABLES

Do left-handed or right-handed shooters score more goals?

TRY

```
merged = skaters.merge(rosters[["player_id", "season", "shoots"]],
                       on=["player_id", "season"], how="left")
merged.groupby("shoots")["goals"].mean()
```

💡 Goals live in **skaters**, handedness lives in **rosters**. **player_id** connects them.

BONUS CHALLENGE

BUILD THE STANDINGS. WHO FINISHES FIRST?

2 points for a win, 1 for an overtime or shootout loss. One row per team-season with GP, W, L, OTL, SOL, PTS, GF, GA and goal differential.

YOUR PANDAS TOOLKIT

Match the question to the tool

Question	pandas
How big is it?	df.shape · df.info() · df.head()
One column	df["col"] · .max() · .mean() · .sum()
Which rows?	df[df["col"] >= 20] · & | · .isin([...])
Top N	df.sort_values("col", ascending=False) · df.nlargest(5, "col")
New column	df["new"] = df["a"] - df["b"]
By group	df.groupby("g")["col"].mean() · .value_counts() · .unstack()
Missing	df.isna().sum() · dropna() · fillna()
Two tables	a.merge(b, on="key", how="left")